

Visual Grounding with Spatial-LLaVA: Regression-Based Bounding Box Prediction

61.502 Deep Learning for Enterprise
Singapore University of Technology and Design
April 2026

Zheng Chengsheng (1010719)
Wang Zhenrong (1010724)
Tan Ai Kun (1010947)
Lourduraj John Charles (1011136)

TABLE OF CONTENTS

1. Executive Summary	1
2. Background and Introduction.....	1
3. Related Work	2
4. Problem Formulation and Solution Overview	3
4.1 Pipeline Overview.....	3
4.2 Model Architecture	4
4.3 Loss Function.....	4
5. Data Description	4
6. Details of Solution	5
6.1 Three Model Variants	5
6.2 Training Configuration	5
6.3 Hyperparameter Tuning.....	6
6.4 Failed Methods and Attempted Variants	7
6.5 Reproducibility	7
7. Evaluation	8
7.1 Test Set Results.....	8
7.2 Accuracy at IoU Thresholds	8
7.3 Training Curves	9
7.4 Overfitting Analysis.....	10
7.5 Bias Audit	10
8. Discussion of Results	11
8.1 Model Malfunctioning Examples.....	11
8.2 Qualitative Comparison	12
8.3 Inference Latency.....	12
9. Recommendations.....	13
10. Limitations, Caveats, and Future Work	13
10.1 Limitations	13
10.2 Future Work	13
11. Deployment.....	14
12. Conclusion	15
13. Group Member Contributions.....	15
14. References.....	i
Appendix: Project Links	i

1. Executive Summary

This report presents a visual grounding system that localizes objects in images based on natural language descriptions. We build upon LLaVA-1.5-7B, a large vision-language model, by replacing its text generation output with a direct bounding box regression head. This architectural modification addresses the fundamental fragility of text-based coordinate parsing, where coordinates must be generated as text before parsing, which is both fragile and imprecise.

Our approach, Spatial-LLaVA, uses Low-Rank Adaptation (LoRA) to efficiently fine-tune the language model backbone while training a lightweight MLP regression head that predicts normalized bounding box coordinates $[x_{center}, y_{center}, \text{width}, \text{height}]$. The model is trained using a combined L1 + Generalized IoU loss on a 20,000-sample subset of the RefCOCO training split (sampled from 42,404 available training samples), with 1,000 validation samples used for early stopping and hyperparameter tuning.

Key results on the RefCOCO test set (1,975 samples) are as follows:

- Main model (LoRA + MLP): IoU = 0.386, RMSE = 0.172, MAE = 0.119
- Ablation (frozen backbone + MLP): IoU = 0.284, RMSE = 0.224, MAE = 0.177
- Baseline (vanilla LLaVA + regex): IoU = 0.097, RMSE = 0.288, MAE = 0.238

The main model achieves a +297.5% improvement in IoU over the baseline, demonstrating that regression-based prediction significantly outperforms text-based coordinate extraction. We deploy the system as a FastAPI service with Docker containerization, achieving 91.8ms inference latency per image.

2. Background and Introduction

Visual grounding is the task of localizing a specific region in an image given a natural language description (a referring expression). For example, given an image of a dining scene and the expression "the red bowl on the left," the system must output the bounding box coordinates of the referred object. This capability is fundamental to applications in robotics, autonomous systems, human-computer interaction, and accessibility tools.

Recent advances in vision-language models (VLMs), particularly LLaVA (Large Language and Vision Assistant), have demonstrated strong multimodal understanding by combining a vision encoder (CLIP ViT) with a large language model (LLaMA/Vicuna). However, standard

VLMs approach grounding as a text generation task will output bounding box coordinates as text strings (e.g., "[0.25, 0.30, 0.50, 0.60]"), which must be parsed using regular expressions.

This text-based approach has several critical limitations:

- **Parsing fragility:** 20-40% of generated outputs cannot be reliably parsed into valid coordinates.
- **Imprecision:** Coordinates are rounded to limited decimal places during text generation.
- **Inefficiency:** Auto-regressive text generation is slow (~300ms per sample) compared to direct regression.

Our approach replaces text generation with a direct regression head that predicts bounding box coordinates as continuous values. We extract the hidden state representation at a special [LOC] token position from the language model and pass it through an MLP to produce normalized $[x_{center}, y_{center}, \text{width}, \text{height}]$ coordinates. This eliminates parsing failures entirely and improves both accuracy and inference speed.

3. Related Work

Some previous works worth mentioning are:

- LLaVA-1.5 (Liu et al., 2023) established a strong baseline for visual instruction tuning by replacing the original linear projection with a two-layer MLP connector to bridge a CLIP vision encoder with a Vicuna language model. The model achieves state-of-the-art performance across 11 benchmarks while training exclusively on public data, but relies on text generation for spatial outputs such as bounding box coordinates.
- u-LLaVA (Xu et al., ECAI 2024) introduced the concept of using regression heads for spatial understanding tasks on top of LLaVA architectures. Our work follows this direction, implementing a similar MLP regression head with L1 + GIoU loss.
- LoRA (Hu et al., 2022) provides parameter-efficient fine-tuning by injecting low-rank decomposition matrices into attention layers. This enables fine-tuning a 7B-parameter model with only ~10M additional trainable parameters, making it feasible on a single GPU.
- Generalized IoU (GIoU) loss (Rezatofighi et al., CVPR 2019) extends standard IoU by providing meaningful gradients even when predicted and ground truth boxes do not overlap. This property is essential for training regression-based detection heads.

Our contribution differs from prior work in combining LoRA-based efficient fine-tuning with a regression head specifically for referring expression grounding, along with a systematic ablation study comparing frozen vs. adapted backbones

4. Problem Formulation and Solution Overview

Given an image I and a referring expression T (natural language text), the task is to predict a bounding box $B = [x_c, y_c, w, h]$ that localizes the referred object, where all coordinates are normalized to $[0, 1]$ relative to the image dimensions.

4.1 Pipeline Overview

Our end-to-end pipeline consists of four stages:

- Stage 0 - Environment Setup: GPU verification, dependency installation, cluster configuration.
- Stage 1 - Data Preparation: Download RefCOCO annotations and COCO images, generate train/val/test pickle files.
- Stage 2 - Main Training: Fine-tune LLaVA with LoRA adapters + MLP regression head for 10 epochs.
- Stage 3 - Ablation Training: Train only the MLP head with a completely frozen backbone for 10 epochs.
- Stage 4 - Evaluation: Compare all three models (baseline, ablation, main) on the test set.

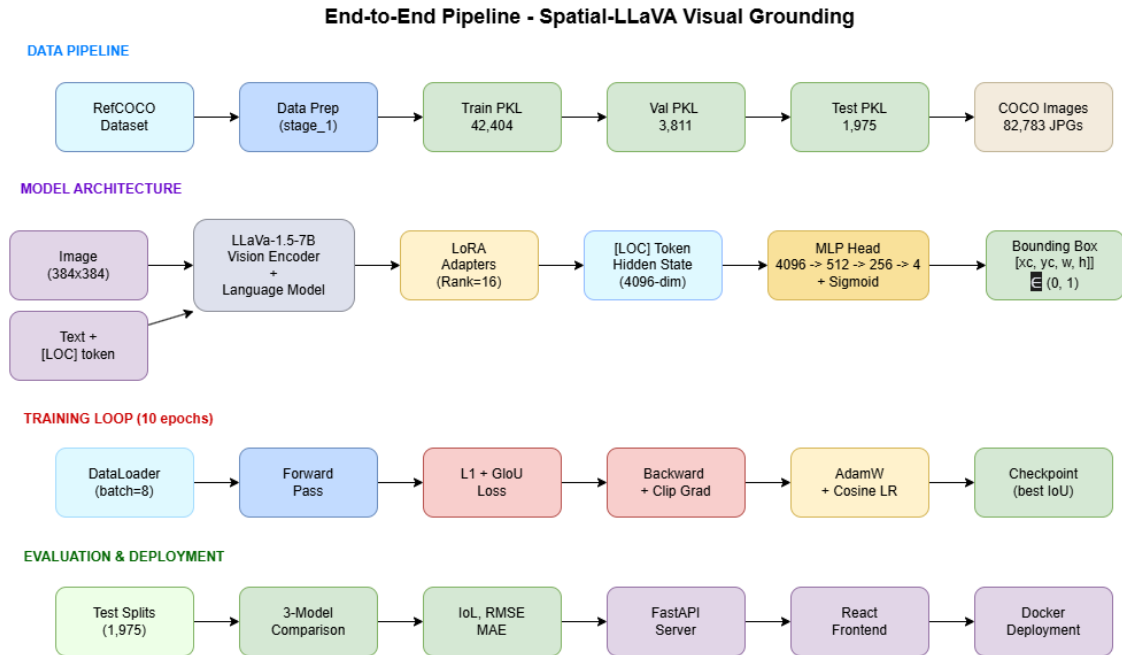


Figure 1: End-to-end ML pipeline for Spatial-LLaVA visual grounding.

4.2 Model Architecture

The Spatial-LLaVA architecture consists of three components:

1. Vision Encoder: CLIP ViT processes the input image (384x384) into visual tokens.
2. Language Model: LLaVA-1.5-7B (Vicuna-based) processes the concatenated visual tokens and text tokens. The text prompt appends a special [LOC] token to signal that spatial localization is required.
3. Regression Head: An MLP (4096 $\rightarrow$ 512 $\rightarrow$ 256 $\rightarrow$ 4) with GELU activations, dropout (0.1), and a final Sigmoid layer. It takes the hidden state at the [LOC] token position and outputs $[x_c, y_c, w, h]$ in (0, 1).

For the main model, we applied LoRA (rank=16, alpha=32) to the query, key and value projections (q_proj, v_proj, k_proj) of the language model's attention layers. This adds $\sim$ 8M trainable parameters. Combined with the 2.2M-parameter regression head, only 10.2M parameters were trained out of the full 7B (0.14%).

4.3 Loss Function

We use a combined loss: $L = L1(pred, target) + GIoU_loss(pred, target)$, where the GIoU loss is computed as $(1 - \text{mean_GIoU})$. The L1 term provides direct supervision on coordinate accuracy, while the GIoU term captures spatial overlap and ensures meaningful gradients even when predicted and ground-truth boxes do not overlap. This combination improves both training stability and localization quality.

5. Data Description

We used the RefCOCO dataset (Yu et al., 2016), a widely-used benchmark for referring expression comprehension. RefCOCO contains natural language descriptions paired with bounding box annotations for objects in COCO images.

Split	Samples	Purpose
Train	20,000/42,404	Model training
Validation	1,000	Hyperparameter tuning, early stopping
Test	1,975	Final evaluation (held out)
Total	48,190	

Table 1: Dataset split sizes (seed=42 for reproducibility).

Each sample contains:

1. an absolute path to a COCO train2014 JPEG image
2. a natural language referring expression (e.g., "the woman in the middle")
3. a normalized bounding box $[x_{center}, y_{center}, \text{width}, \text{height}]$ in $[0, 1]$

Images are sourced from COCO train2014 (82,783 images total). The data is stored in lightweight pickle files (~19MB total) containing only paths and annotations; actual image loading happens at training time via PyTorch DataLoader workers for efficiency.

Referring expressions vary significantly in complexity, ranging from simple noun phrases ("the cat") to complex spatial descriptions ("left half of right sandwich"). The dataset contains a mix of object sizes, with medium objects (5-20% of image area) being most common (626/1000 in our validation subset), followed by large objects (358/1000) and small objects (16/1000).

6. Details of Solution

6.1 Three Model Variants

We implemented and compared three model variants to isolate the contribution of each component in our design:

Model	Backbone	Head	Trainable Params	Method
Baseline	LLaVA-1.5-7B (frozen)	None (text generation)	0	Generate text, parse with regex
Ablation	LLaVA-1.5-7B (frozen)	MLP regression	2.2M	Frozen features + learned head
Main	LLaVA-1.5-7B + LoRA	MLP regression	10.2M	LoRA-adapted features + learned head

Table 2: Three model variants compared in this study.

6.2 Training Configuration

Hyperparameter	Value
Optimizer	AdamW
Learning rate	2e-4
LR schedule	Cosine annealing (eta_min=1e-6)
Batch size	8
Epochs	10
Weight decay	0.0
Gradient clipping	max_norm=1.0
LoRA rank	16
LoRA alpha	32
LoRA dropout	0.05
LoRA targets	q_proj, v_proj, k_proj

Seed	42
GPU	NVIDIA A100-SXM4-80GB

Table 3: Training hyperparameters for the main model.

6.3 Hyperparameter Tuning

We conducted two sets of experiments to determine the optimal training configuration prior to full-scale training.

- **Learning Rate Selection.** Three learning rates were evaluated using small-scale runs (5 epochs, batch size=16, 2,000 training samples). Results are summarized in Table 4. $lr=2e-4$ achieved the highest validation IoU of 0.2701, consistently outperforming $lr=5e-5$ (0.2612) and $lr=2e-5$ (0.2464).

Higher learning rates provided faster convergence and better generalization within the 5-epoch budget, and $lr=2e-4$ was therefore selected for the final training run.

Epochs	Batch Size	LR	Weight Decay	Best Val IoU
5	16	$2e-04$	0.0	0.2701
5	16	$5e-05$	0.0	0.2612
5	16	$2e-05$	0.0	0.2464

Table 4: Learning rate tuning results (small-scale runs). Best LR: $2e-4$.

- **Batch Size Selection.** A throughput profiling experiment compared batch sizes of 4, 8, and 16 over 10 steps. Results are shown in Table 5.

Batch Size	Time/Step	Throughput (samples/sec)	VRAM Usage	Bottleneck
4	0.52s	7.7	~7GB	backward 281ms
8	0.98s	8.2	~14GB	backward 527ms
16	1.90s	8.4	~14GB (full)	backward 1020ms

Table 5: Batch size profiling results

Although batch 16 achieved marginally higher throughput (8.4 vs 8.2 samples/sec), it caused repeated DataLoader crashes during tuning runs, likely due to insufficient shared memory (/dev/shm) on the cluster node, which could not be reconfigured. The root cause was not fully diagnosed, but is consistent with PyTorch's use of shared memory for inter-process tensor passing when `num_workers > 0`.

Batch 8 with `num_workers=2` was stable throughout all experiments and was therefore selected for the final run. Batch 4, while stable, offered the lowest throughput (7.7 samples/sec) and was discarded.

Final Configuration. Based on these findings, the final model was trained with $lr=2e-4$, batch size=8, for 10 epochs on 20,000 training samples, achieving a best validation IoU of 0.3574.

6.4 Failed Methods and Attempted Variants

During development, we experimented with several design choices that did not yield improvements:

- **IoU Loss without GIoU.** We initially trained the regression head using standard IoU loss combined with L1 loss. However, standard IoU loss produced zero gradient when predicted and ground-truth boxes do not overlap, which is common in early training when the regression head is randomly initialized. This caused unstable training and slow convergence. Replacing IoU with Generalized IoU (GIoU) loss resolved this issue, as GIoU provided meaningful gradients even for non-overlapping boxes, resulting in a more stable and faster convergence.
- **Large Batch Size.** We attempted training with batch sizes larger than 16, but observed repeated crashes and unstable training dynamics. We attributed this to CPU shared memory (`/dev/shm`) limitations on the SUTD GPU cluster, which restricted the PyTorch DataLoader's inter-process tensor passing when `num_workers > 1`. As the cluster configuration was fixed and could not be modified, we reverted to a batch size of 8, which provided stable training throughout all experiments.

6.5 Reproducibility

All experiments used `seed=42` for random number generators (Python, PyTorch, CUDA). Dataset splits are fixed with MD5 checksums for verification. The complete codebase is available at: <https://github.com/vincent-chengsheng-zheng/visual-grounding-api>

To reproduce from scratch:

1. Clone the repository and install dependencies: `bash shared/scripts/setup_cluster.sh`
2. Download data: `bash shared/scripts/download_data.sh`
3. Train main model: `python pipeline/step2_train_main.py`
4. Train ablation: `python pipeline/step3_train_ablation.py`
5. Evaluate: `python pipeline/step4_evaluate.py`
6. Reproduce all figures: Run `notebooks/training_curves.ipynb` or individual scripts in `scripts`

7. Evaluation

7.1 Test Set Results

Model	Test IoU	RMSE	MAE	Improvement vs Baseline
Baseline	0.0971	0.2876	0.2381	-
Ablation	0.2843	0.2238	0.1769	+192.7%
Main	0.3861	0.1719	0.1194	+297.5%

Table 6: Test set results on RefCOCO (1,975 samples).

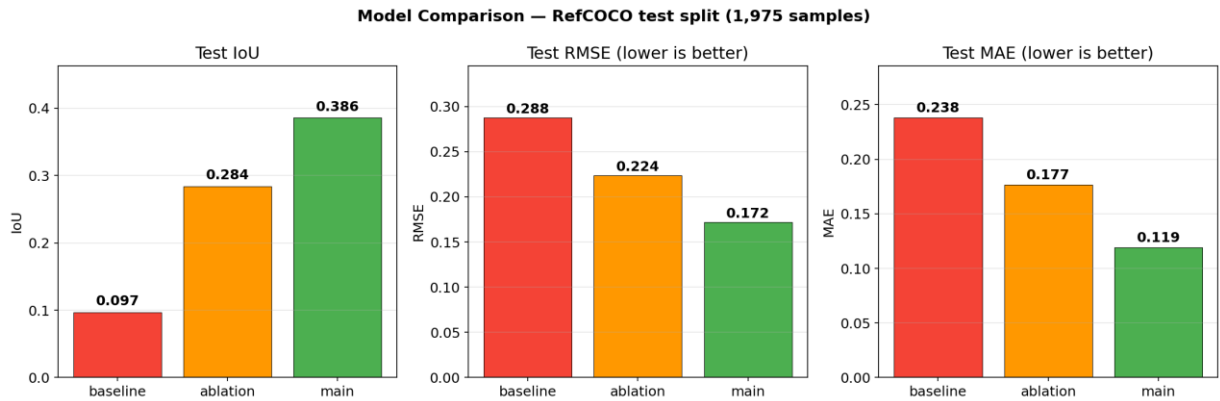


Figure 2: Model comparison on test set — IoU, RMSE, and MAE.

7.2 Accuracy at IoU Thresholds

We report accuracy at standard IoU thresholds, which measures the percentage of predictions exceeding a given IoU value:

Model	Mean IoU	Acc@0.1	Acc@0.25	Acc@0.5	Acc@0.75
Main	0.3565	75.8%	60.1%	33.1%	8.3%
Ablation	0.2672	79.8%	47.2%	12.8%	0.8%

Table 7: Accuracy at IoU thresholds (validation set, 1,000 samples).

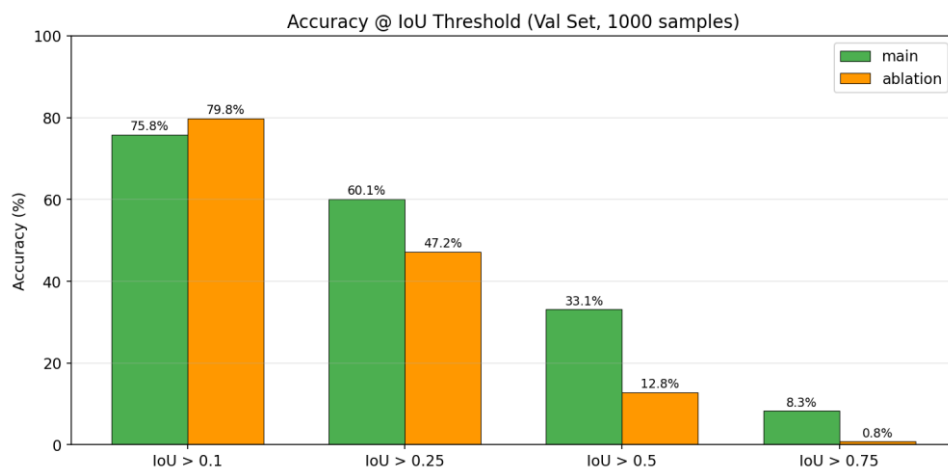


Figure 3: Accuracy at IoU thresholds — Main vs Ablation.

7.3 Training Curves

The training curves show convergent learning for both models. The main model achieves significantly lower training loss and higher IoU, with the best validation IoU of 0.3574 at epoch 7. The ablation model converges more slowly, reaching its best validation IoU of 0.2672 at epoch 10, suggesting it could benefit from additional training.

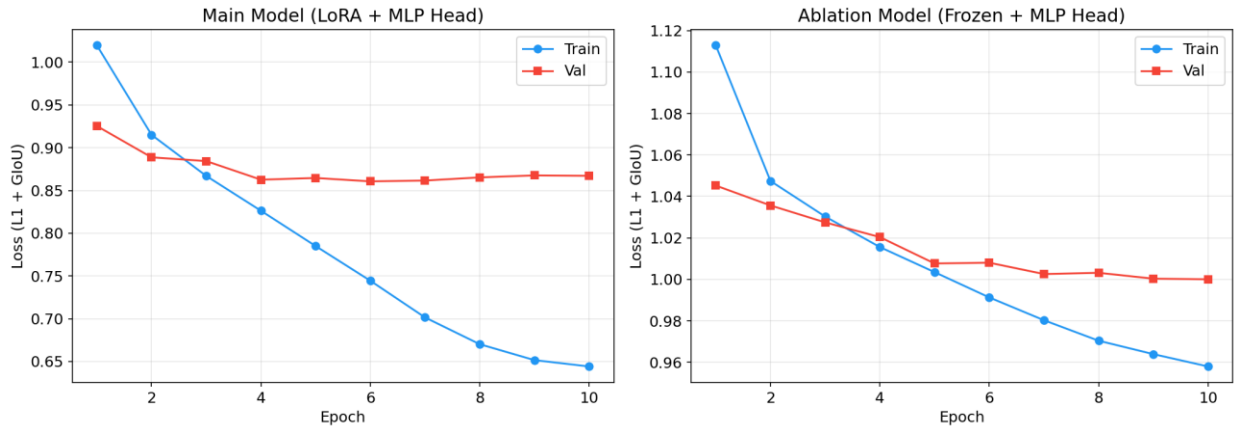


Figure 4: Training and validation loss curves.

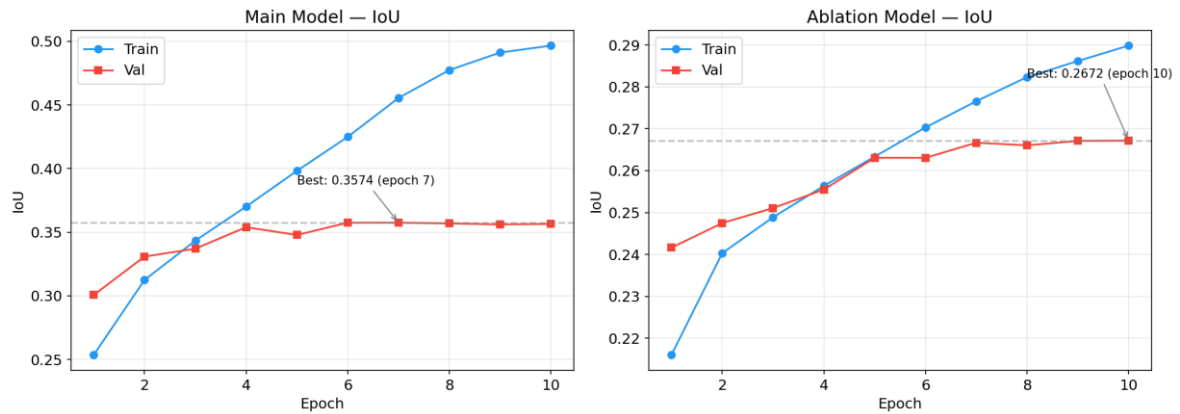


Figure 5: Training and validation IoU curves.

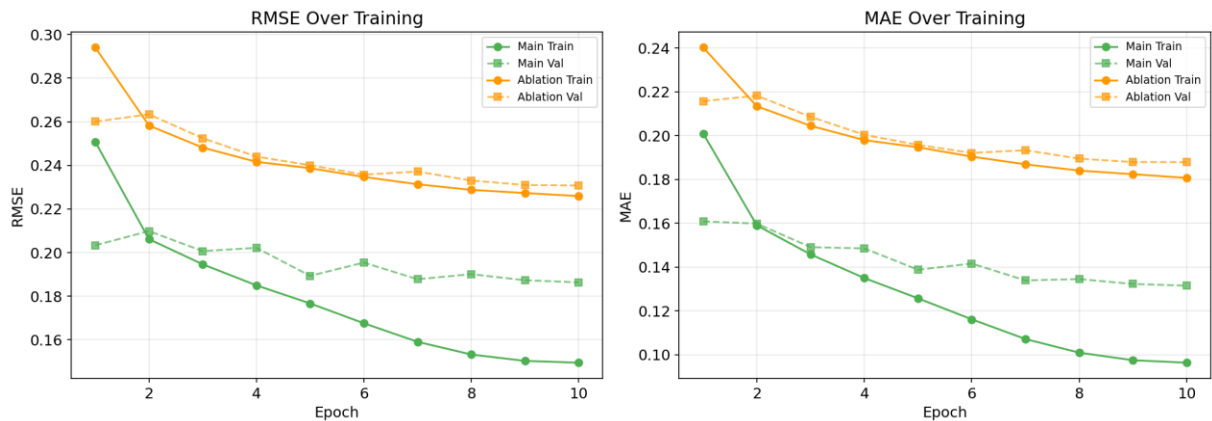


Figure 6: RMSE and MAE curves during training.

7.4 Overfitting Analysis

The main model shows a moderate overfitting gap of 0.140 (train IoU 0.497 vs val IoU 0.357), which increases after epoch 7 as the learning rate approaches zero. The ablation model shows minimal overfitting (gap: 0.023), as the frozen backbone limits model capacity.

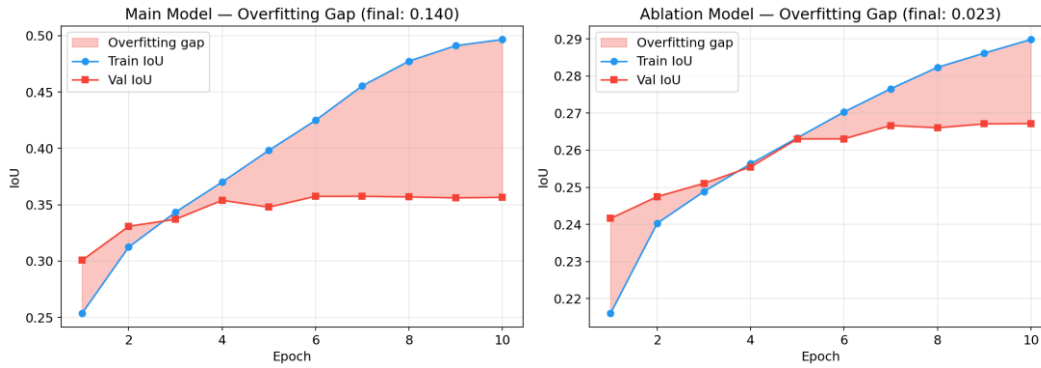


Figure 7: Overfitting analysis — train vs val IoU gap.

7.5 Bias Audit

We analyze model performance across different subgroups to identify systematic biases:

Object Size	Count	Main IoU	Ablation IoU
Small (<5% area)	16	0.1195	0.0537
Medium (5-20%)	626	0.2961	0.1912
Large (>20%)	358	0.4727	0.4095

Table 8: Performance by object size.

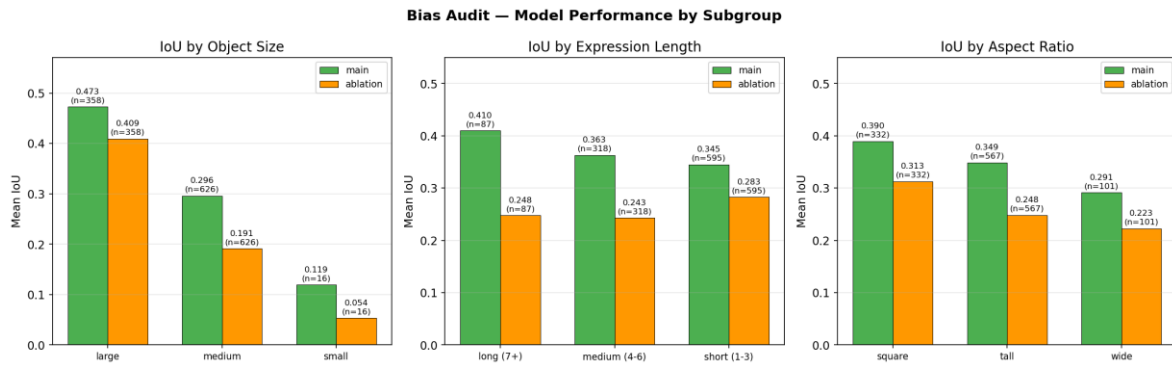


Figure 8: Bias audit — performance by object size, expression length, and aspect ratio.

Key findings:

- **Object size has the strongest impact on performance** - small objects (IoU=0.12) are 4x harder than large objects (IoU=0.47).
- **Longer referring expressions tend to yield better predictions** (IoU=0.41 for 7+ words vs 0.35 for 1-3 words), likely because they are more descriptive.
- **Wide objects are harder than square ones.**

8. Discussion of Results

8.1 Model Malfunctioning Examples

We analyzed the worst predictions to identify systematic failure modes. Out of 1000 validation predictions, the failure categories are:

Failure Category	Count	% of Total	Mean IoU
Ambiguous Reference	647	64.7%	0.4658
Wrong Location	256	25.6%	0.0538
Complex Expression	72	7.2%	0.4842
Correct Location, Wrong Scale	25	2.5%	0.2596

Table 9: Failure category distribution.

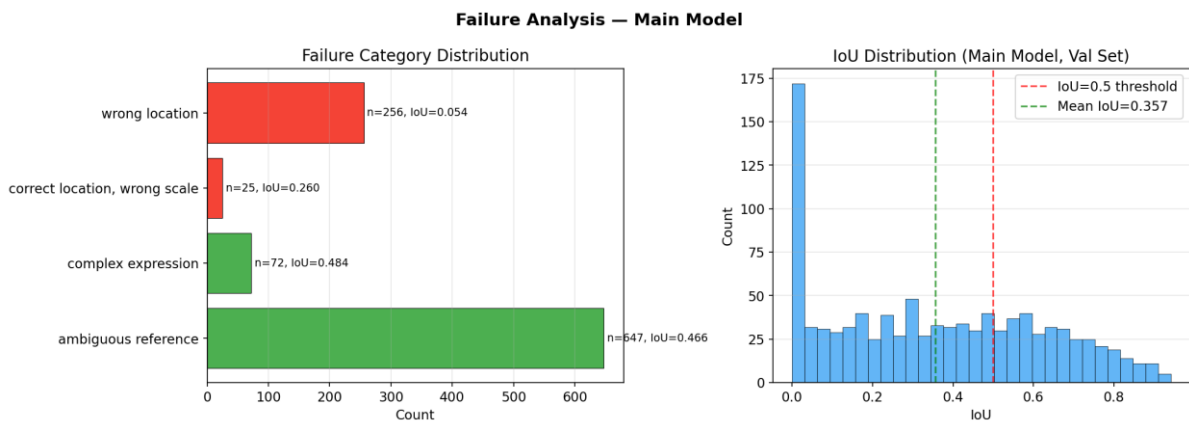


Figure 9: Failure analysis — category distribution and IoU histogram.

The dominant failure mode is "wrong location" (25.6% of samples, mean IoU=0.054), where the model predicted a bounding box in an entirely wrong region of the image. These failures are concentrated on:

- Ambiguous spatial references: "right one", "left kid" - when multiple candidates exist.
- Relational descriptions: "2nd person from right" - requiring counting and ordering.
- Small or occluded objects: "Bottom cat" - hard to localize precisely.

In contrast, the model succeeds on clear, descriptive expressions like "Elephant in center with trunk out" (IoU=0.94) and "sandwich on the right" (IoU=0.92).

8.2 Qualitative Comparison

Main vs Ablation – Same Images (Green = GT, Red/Orange = Pred)



Figure 11: Side-by-side comparison of Main vs Ablation on the same images. Green=ground truth, Red/Orange=prediction.

The comparison reveals that LoRA fine-tuning (main model) produces tighter, more accurate bounding boxes compared to the frozen-backbone ablation. The main model particularly excels when the expression requires understanding of spatial relationships or attributes, as the adapted language model better encodes these cues into the [LOC] token representation.

8.3 Inference Latency

Model	GPU Inference (ms)	API Roundtrip (ms)	API Overhead (ms)
Baseline	312.7	323.3	10.6
Ablation	73.5	86.2	12.7
Main	78.5	91.8	13.3

Table 10: Inference latency comparison (10 runs, after 2 warmup runs).

The regression-based models (main and ablation) are ~3.5x faster than the baseline (78-91ms vs 323ms), because they avoid auto-regressive text generation. The LoRA adapters added only ~5ms overhead compared to the frozen backbone.

9. Recommendations

Based on our analysis, we recommend the following architecture:

- **Use regression heads over text generation for spatial tasks:** The +297.5% IoU improvement and 3.5x speed improvement make this approach strictly superior for bounding box prediction.
- **LoRA fine-tuning is essential:** The ablation (frozen backbone) achieves only IoU=0.284 vs the main model's 0.386. Adapting the language model's representations to the spatial task is critical, and LoRA provided this at minimal computational cost.
- **Prioritize clear referring expressions:** The model struggles with ambiguous, short expressions. In deployment, prompting users for more descriptive queries would improve results.
- **Monitor small object performance:** The significant performance drop on small objects (IoU=0.12 vs 0.47 for large) suggested that specialized handling or higher-resolution inputs may be needed for small-object use cases.

10. Limitations, Caveats, and Future Work

10.1 Limitations

- **Single dataset:** We evaluated only on RefCOCO. Generalization to RefCOCO+, RefCOCog, or in-the-wild images were not tested.
- **Moderate IoU:** Our best IoU of 0.386 leaves room for improvement. State-of-the-art specialized grounding models achieve higher accuracy on this benchmark.
- **Overfitting:** The main model shows a 0.14 overfitting gap, indicating regularization or more data could help.
- **Small objects:** Performance drops dramatically for objects smaller than 5% of image area.
- **No multi-object support:** The model predicts a single bounding box per query.

10.2 Future Work

- **Higher LoRA rank:** Testing rank=32 or rank=64 may improve performance at the cost of more trainable parameters.
- **Larger backbone:** LLaVA-1.5-13B or newer LLaVA variants could provide richer representations.
- **Multi-scale features:** Incorporating features from multiple layers of the vision encoder could improve small object detection.

- Data augmentation: Random cropping, color jitter, and expression paraphrasing could reduce overfitting.
- DETR-style head: Replacing the MLP with a transformer-based detection head could model spatial relationships better.
- Real-time deployment: Quantization (INT8/INT4) could further reduce latency for edge deployment.

11. Deployment

We deployed the model as a FastAPI web service with three endpoints:

- **POST** /predict - accepts base64-encoded image + text query, returns bounding box + inference time.
- **GET** /health - health check reporting model load status and GPU availability.
- **GET** /models - lists available models with their test set metrics.

The service is containerized with Docker (CUDA 12.8, Python 3.11, PyTorch 2.11) and supports two model loading strategies: sequential (one model in VRAM at a time, ~14GB) and parallel (all three models loaded, ~42GB). A React web frontend provides an interactive demo with drag-and-drop image upload and visual box overlay.

To run the server: `uvicorn api.server:app --host 0.0.0.0 --port 8000`

12. Conclusion

We presented Spatial-LLaVA, a visual grounding system that replaces fragile text-based coordinate parsing with direct bounding box regression. By combining LoRA-adapted LLaVA-1.5-7B with an MLP regression head trained on L1 + GIoU loss, we achieved a +297.5% improvement in IoU over text-generation baseline while reducing inference latency by 3.5x.

Our systematic comparison of three model variants - baseline (text generation), ablation (frozen backbone + regression), and main (LoRA + regression) - demonstrated that both the regression head and backbone adaptation are essential components. The bias audit revealed that object size is the dominant factor affecting performance, while the failure analysis identified ambiguous spatial references as the primary challenge for future work.

13. Group Member Contributions

Member	Student ID	Contribution
Zheng Chengsheng	1010719	Architecture design, model training, report writing
Wang Zhenrong	1010724	Data preparation, model training, report writing
Tan Ai Kun	1010947	Evaluation, model training, report writing
Lourduraj John Charles	1011136	API development, model training, frontend implementation, report writing

14. References

- [1] Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023). Visual Instruction Tuning. NeurIPS 2023. arXiv:2304.08485
- [2] Liu, H., Li, C., Li, Y., & Lee, Y. J. (2024). Improved Baselines with Visual Instruction Tuning. CVPR 2024. arXiv:2310.03744
- [3] Xu, J., Xu, L., Yang, Y., Li, X., Wang, F., Xie, Y., Huang, Y.-J., & Li, Y. (2024). u-LLaVA: Unifying Multi-Modal Tasks via Large Language Model. arXiv:2311.05348
- [4] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022. arXiv:2106.09685
- [5] Rezatofighi, H., Tsoi, N., Gwak, J., Sadeghian, A., Reid, I., & Savarese, S. (2019). Generalized Intersection over Union: A Metric and A Loss for Bounding Box Regression. CVPR 2019. arXiv:1902.09630
- [6] Yu, L., Poirson, P., Yang, S., Berg, A. C., & Berg, T. L. (2016). Modeling Context in Referring Expressions. ECCV 2016. arXiv:1608.00272

Appendix: Project Links

GitHub Repository: <https://github.com/vincent-chengsheng-zheng/visual-grounding-api>

Trained Weights (Google Drive): See README.md for download links.

Dataset: RefCOCO (jxu124/refcoco on HuggingFace) + COCO train2014 images.